🚀 Android Moderno: Pruebas y Firma de Apps

De Cero a Producción con Kotlin 2.0 y Jetpack Compose

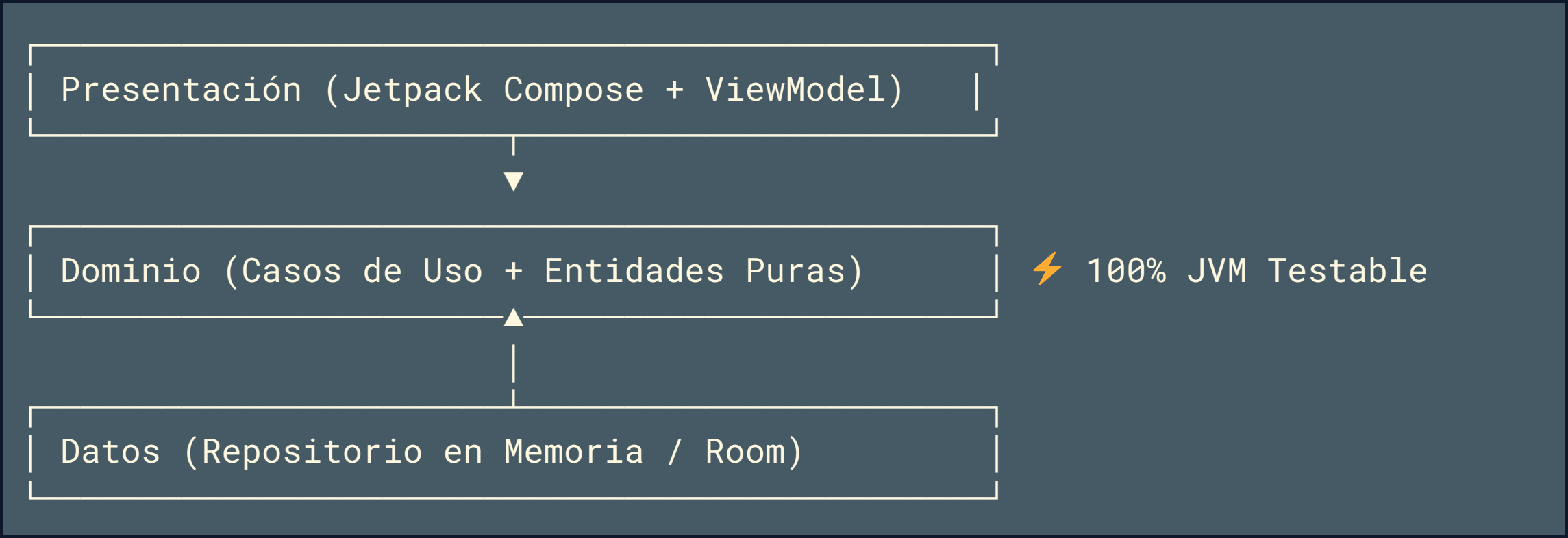
Guía Docente y Práctica para Estudiantes

📋 Agenda de la Sesión

1. **Arquitectura Limpia & Kotlin 2.0** (Estructura de la app)
2. **Cómo Gestionar Dependencias** (`libs.versions.toml` sin enredos)
3. **Pruebas Unitarias** (`AddTodoUseCaseTest`)
4. **Pruebas End-to-End (E2E)** con Compose (`TodoAppE2ETest`)
5. **Criptografía & Firma de Apps** (Keystores, Schemes v1-v4 y CI/CD)
6. **Taller Práctico** (Compilación y verificación en vivo)

🏗️ 1. Arquitectura de la App Todo

Separación de responsabilidades clara:



> **Regla de Oro:** El Dominio no conoce a Android. Esto permite que

📦 2. Dependencias de Testing en Android

¿Dónde y cómo se definen las librerías hoy?

- En `gradle/libs.versions.toml`:

```
[versions]
junit = "4.13.2"
turbine = "1.2.0"

[libraries]
junit = { group = "junit", name = "junit", version.ref = "junit" }
turbine = { group = "app.cash.turbine", name = "turbine", version.ref = "turbine" }
```

- En `app/build.gradle.kts`:
  - `testImplementation(...)` -> Pruebas Unitarias (JVM local)
  - `androidTestImplementation(...)` -> Pruebas Instrumentadas /

🧪 3. Pruebas Unitarias: El Caso de Uso

Objetivo: Verificar las reglas de negocio antes de tocar la interfaz gráfica.

```
@Test
fun `invoke with blank title fails with descriptive error`() = runTest {
    // 1. Arrange: Título vacío
    val blankTitle = "   "

    // 2. Act: Ejecutamos el caso de uso
    val result = useCase(title = blankTitle)

    // 3. Assert: Comprobamos el fallo controlado
    assertTrue(result.isFailure)
    assertEquals("Task title cannot be empty", result.exceptionOrNull()?.message)
    assertTrue(repository.getTodos().first().isEmpty())
}
```

🤖 4. Pruebas E2E: El Viaje del Usuario

Objetivo: Simular lo que ve y toca una persona en su pantalla.

- Levanta la `MainActivity` real:

```
@get:Rule
val composeTestRule = createAndroidComposeRule<MainActivity>()
```

- Pasos del test interactivo:
  1. Pulsar botón flotante `+` (`performClick()`)
  2. Escribir título y descripción (`performTextInput(...)`)
  3. Guardar y verificar que aparece en el listado (`assertIsDisplayed()`)

🔐 5. ¿Por qué firmamos las aplicaciones Android?

1. **Autenticidad:** Garantiza la identidad criptográfica del autor.
2. **Integridad:** El sistema operativo comprueba que ningún byte del APK fue modificado tras compilar.
3. **Actualizaciones:** Android **bloquea** cualquier actualización si la clave de firma no coincide exactamente con la ya instalada.

🔑 6. Anatomía de un Keystore (Almacén de Claves)

- **Formato estándar:** PKCS12 (archivo `.keystore` o `.p12`).
- **Algoritmo:** RSA 2048 bits con firma SHA-256 / SHA-384.
- **Componentes:**
  - Contraseña del almacén (`storePassword`)
  - Alias de la clave privada (`keyAlias`)
  - Contraseña de la clave privada (`keyPassword`)
- **Huella Digital (SHA-256 Fingerprint):** El código único que se registra en Firebase o Google Cloud Console.

🛡️ 7. Esquemas de Firma en Android

Android ha evolucionado en 4 esquemas principales:

- **v1 (JAR Signing):** Firma archivo por archivo dentro del ZIP.
- **v2 (APK Signature):** Firma el archivo binario completo. Rápido y seguro.
- **v3 (Key Rotation):** Permite cambiar de clave histórica si se compromete.
- **v4 (Streaming):** Permite instalar apps gigantes por streaming sin esperar a descargar el 100%.

⚙️ 8. Configuración Segura en Gradle

En `app/build.gradle.kts`:

```
signingConfigs {
    create("release") {
        storeFile = rootProject.file("keystore/debug-demo.keystore")
        storePassword = "demo1234"
        keyAlias = "demo_key"
        keyPassword = "demo1234"
        enableV1Signing = true
        enableV2Signing = true
        enableV3Signing = true
    }
}
```

> **Consejo de Seguridad:** En producción, estas contraseñas se leen

🛠️ 9. Manos a la Obra: Comandos Clave

- **Ejecutar Pruebas Unitarias:**

```
./gradlew test
```

- **Compilar APK Release Firmado:**

```
./gradlew assembleRelease
```

- **Verificar la Firma del APK generado:**

```
./scripts/verify-signature.sh app/build/outputs/apk/release/app-release.apk
```

🎯 Conclusión y Preguntas

- Arquitectura limpia = Pruebas fáciles y rápidas.
- Los tests unitarios cuidan la lógica de negocio; los tests E2E cuidan la experiencia del usuario.
- La firma digital es el candado de seguridad inmutable de todo el ecosistema Android.

¡A programar! 💻